ImMAP: guided posterior ascent,
with and without a bridge

ImMAP project notes

November 2026

Abstract

We want the MAP estimate of a target image $\boldsymbol{x}_0$ given the side information available: a guide image $\boldsymbol{x}_1$ (a paired scan, typically of another contrast), a measurement $\boldsymbol{y}$, or both. Revealing the side information one variable at a time splits the log-posterior exactly into the target prior plus one likelihood term per variable. The split survives Gaussian smoothing, and by Tweedie's formula the gradient of each term is a difference of conditional expectations. One annealed ascent step therefore serves every conditioning set; ImMAP2 is a case study. A denoiser must be evaluated at every smoothing level, including very large ones, and that is where a bridge helps. It feeds the network a bridge sample that carries the same information as a very noisy target but stays bounded. I^{2}SB is the ascent run through this bridge preconditioner, and each line of its sampler is a reparameterised line of the ascent.

Setting. Real vectors in $\mathbb{R}^n$; the complex case is identical with Wirtinger gradients. Pairs $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ have a joint law. The measurement is $\boldsymbol{y} = \boldsymbol{E}\boldsymbol{x}_0 + \boldsymbol{n}$, with $\boldsymbol{n} \sim \mathcal{N}(\boldsymbol{0}, \sigma_n^2 \boldsymbol{I})$ independent of $(\boldsymbol{x}_0, \boldsymbol{x}_1)$, so $\boldsymbol{y}$ and $\boldsymbol{x}_1$ are conditionally independent given $\boldsymbol{x}_0$. The noisy target is $\boldsymbol{u} = \boldsymbol{x}_0 + \sigma\boldsymbol{\varepsilon}$, with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ independent of everything else. For a set $\boldsymbol{c} \subseteq \{\boldsymbol{x}_1, \boldsymbol{y}\}$ of conditioning variables,

$$D_{\boldsymbol{c}}(\boldsymbol{u}, \sigma) \coloneqq \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \boldsymbol{c}] \quad \text{at noise level } \sigma.$$

1 The objective

Given side information $\boldsymbol{c}$, the goal is

$$\hat{\boldsymbol{x}}_0 = \arg\max_{\boldsymbol{x}_0} \ \log p(\boldsymbol{x}_0 \mid \boldsymbol{c}), \qquad \boldsymbol{c} \in \{\varnothing,\ \{\boldsymbol{x}_1\},\ \{\boldsymbol{y}\},\ \{\boldsymbol{x}_1, \boldsymbol{y}\}\}. \tag{1}$$

We always condition on every independent source of information available. A variable that is a function of others adds nothing. For example, the zero-filled image $\boldsymbol{E}^H\boldsymbol{y}$ is a function of $\boldsymbol{y}$, so conditioning on it as well as on $\boldsymbol{y}$ is conditioning on $\boldsymbol{y}$. A guide scan is not, so $p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y}) \neq p(\boldsymbol{x}_0 \mid \boldsymbol{y})$ in general.

2 Decomposing along a chain

Reveal the variables of $\boldsymbol{c}$ one at a time, in some order. With two variables, guide first:

$$\log p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y}) = \log p(\boldsymbol{x}_0) + \underbrace{\log \frac{p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)}{p(\boldsymbol{x}_0)}}_{\mathrm{PMI}(\boldsymbol{x}_0; \boldsymbol{x}_1)} + \underbrace{\log \frac{p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y})}{p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)}}_{\mathrm{PMI}(\boldsymbol{x}_0; \boldsymbol{y} \mid \boldsymbol{x}_1)} \tag{2}$$

$$= \log p(\boldsymbol{x}_0) + \log p(\boldsymbol{x}_1 \mid \boldsymbol{x}_0) + \log p(\boldsymbol{y} \mid \boldsymbol{x}_0) + \text{const}. \tag{3}$$

(2) is an identity, because the ratios telescope. Each pointwise mutual information (PMI) term measures how much the newly revealed variable says about $\boldsymbol{x}_0$, given what was revealed before. In (3) each becomes a likelihood:

- by Bayes, $\mathrm{PMI}(\boldsymbol{x}_0; \boldsymbol{a} \mid \boldsymbol{b}) = \log p(\boldsymbol{a} \mid \boldsymbol{x}_0, \boldsymbol{b}) - \log p(\boldsymbol{a} \mid \boldsymbol{b})$, and the last term does not depend on $\boldsymbol{x}_0$;
- by conditional independence, $p(\boldsymbol{y} \mid \boldsymbol{x}_0, \boldsymbol{x}_1) = p(\boldsymbol{y} \mid \boldsymbol{x}_0)$.

So the objective is

$$\textit{target prior} + \textit{guide likelihood} + \textit{measurement likelihood},$$

whichever order the variables are revealed in, and dropping a variable drops its term. With one variable the chain has one step; with none, (1) is the prior alone.

The order does not change the objective. It changes the intermediate conditional expectations that appear below, that is, which estimators the gradient is assembled from. Guide-first uses $D_\varnothing, D_{\boldsymbol{x}_1}, D_{\boldsymbol{x}_1,\boldsymbol{y}}$; measurement-first uses $D_\varnothing, D_{\boldsymbol{y}}, D_{\boldsymbol{x}_1,\boldsymbol{y}}$.

3 Smoothing, and the gradient

As in ImMAP [1], we ascend a Gaussian smoothing of the objective while lowering the smoothing. Let

$$J_\sigma(\boldsymbol{u}) \coloneqq \log p_\sigma(\boldsymbol{u} \mid \boldsymbol{c}), \qquad p_\sigma(\boldsymbol{u} \mid \boldsymbol{c}) = \int \mathcal{N}(\boldsymbol{u}; \boldsymbol{x}, \sigma^2 \boldsymbol{I})\, p(\boldsymbol{x} \mid \boldsymbol{c})\, d\boldsymbol{x}.$$

So J_σ is the log-density of the noisy target given $\boldsymbol{c}$, and $J_\sigma \to \log p(\cdot \mid \boldsymbol{c})$ as $\sigma \to 0$.

Proposition 1 (The chain survives smoothing; each term is a difference of expectations)**.** *Along the chain $\varnothing \subset \boldsymbol{c}_1 \subset \cdots \subset \boldsymbol{c}_K = \boldsymbol{c}$,*

$$J_\sigma(\boldsymbol{u}) = \log p_\sigma(\boldsymbol{u}) + \sum_{k=1}^{K} \log \frac{p_\sigma(\boldsymbol{u} \mid \boldsymbol{c}_k)}{p_\sigma(\boldsymbol{u} \mid \boldsymbol{c}_{k-1})}, \qquad \sigma^2 \nabla J_\sigma(\boldsymbol{u}) = \underbrace{D_\varnothing + \sum_{k=1}^{K} (D_{\boldsymbol{c}_k} - D_{\boldsymbol{c}_{k-1}})}_{= D_{\boldsymbol{c}}(\boldsymbol{u},\sigma)} - \boldsymbol{u}, \quad (4)$$

and each ratio is the PMI between the noisy *target $\boldsymbol{u}$ and the k-th variable.*

Proof. The telescoping is the same as in (2), with $\boldsymbol{u}$ in place of $\boldsymbol{x}_0$. Bayes applies to $(\boldsymbol{u}, \boldsymbol{c}_k)$ because $\boldsymbol{\varepsilon}$ is independent of the side information. Tweedie's formula, $\sigma^2 \nabla \log p_\sigma(\boldsymbol{u} \mid \boldsymbol{c}_k) = D_{\boldsymbol{c}_k}(\boldsymbol{u}, \sigma) - \boldsymbol{u}$, holds for any such $\boldsymbol{c}_k$ (differentiate the convolution under the integral). Apply it to every term; the $-\boldsymbol{u}$'s cancel inside each ratio. □

What a variable contributes to the gradient is how far revealing it moves the conditional expectation, $D_{\boldsymbol{c}_k} - D_{\boldsymbol{c}_{k-1}}$. It vanishes when the variable is uninformative given the others. Summed, the chain gives back $D_{\boldsymbol{c}}$, so an estimator of $D_{\boldsymbol{c}}$ itself needs nothing else. The decomposition earns its keep when $D_{\boldsymbol{c}}$ is *not* directly available. Then one estimates a less-conditioned expectation and adds the last increment separately. That is exactly what a data-consistency step does,

$$D_{\boldsymbol{x}_1,\boldsymbol{y}} = D_{\boldsymbol{x}_1} + \underbrace{(D_{\boldsymbol{x}_1,\boldsymbol{y}} - D_{\boldsymbol{x}_1})}_{\text{measurement term}},$$

with the increment estimated from the measurement model (Section 5).

The smoothed terms are not the unsmoothed likelihoods at a noisy image. The measurement term, for instance, involves $p(\boldsymbol{y} \mid \boldsymbol{u}, \boldsymbol{x}_1) = \int p(\boldsymbol{y} \mid \boldsymbol{x})\, p(\boldsymbol{x} \mid \boldsymbol{u}, \boldsymbol{x}_1)\, d\boldsymbol{x}$: the likelihood averaged over the targets the noisy state still allows. It is weak at large σ and sharpens to $p(\boldsymbol{y} \mid \boldsymbol{x}_0)$ as $\sigma \to 0$.

4 The ascent

The step. Gradient ascent on J_σ with step size $h\sigma^2$ is, by (4),

$$\boxed{\boldsymbol{u}^+ = \boldsymbol{u} + h\sigma^2 \nabla J_\sigma(\boldsymbol{u}) = (1-h)\,\boldsymbol{u} + h\,\boldsymbol{v}, \qquad \boldsymbol{v} = D_{\boldsymbol{c}}(\boldsymbol{u}, \sigma).} \tag{5}$$

Tweedie's second-order identity gives $-\nabla^2 J_\sigma = (\sigma^2 \boldsymbol{I} - \mathrm{Cov}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \boldsymbol{c}])/\sigma^4 \preceq \sigma^{-2}\boldsymbol{I}$. So every $h \in (0, 2)$ increases J_σ, by the descent lemma, whatever $\boldsymbol{c}$ is.

The continuation. After each step, fresh noise is injected so that the iterate is again a noisy target, at the next, smaller level $\sigma' = r\sigma$. An exact step leaves noise $(1-h)\sigma$, so the injection is

$$\gamma^2 = \sigma'^2 - (1-h)^2\sigma^2, \qquad \text{which needs } |1-h| \le r. \tag{6}$$

There are two ways to set r:

- ImMAP parameterises the contraction as $r = 1 - \beta h$ with $\beta \in (0, 1]$, which gives $\gamma = \sigma\sqrt{(1-\beta h)^2 - (1-h)^2}$;
- a fixed ladder $\sigma_N > \cdots > \sigma_0 = 0$ sets $r_k = \sigma_{k-1}/\sigma_k$ and leaves $h_k \in [1 - r_k, 1]$ free.

Algorithm 1 Annealed posterior ascent for $\max_{\boldsymbol{x}_0} \log p(\boldsymbol{x}_0 \mid \boldsymbol{c})$

Require: an estimator of $D_{\boldsymbol{c}}$, direct or assembled along a chain (4)
Require: a start $\boldsymbol{u}$ at level σ ($\sigma = \infty$ allowed); a rule for h, r with $|1-h| \le r$; $\sigma_{\min}$

```
while σ > σ_min do
    v ← D_c(u, σ)                                   ▷ = u + σ² ∇J_σ(u)
    choose h and r
    u ← (1 − h) u + h v + σ √(r² − (1 − h)²) z,  z ∼ N(0, I)      ▷ (5), (6)
    σ ← r σ
end while
return v
```

Three step sizes are worth naming:

- $h = 1 - r^2$ removes the variance $\sigma^2 - \sigma'^2$ and injects $\gamma^2 = \sigma'^2(1 - r^2)$. This is the VE ancestral sampler [4] ($\beta = 1/(1+r)$).
- $h = 1 - r$ injects nothing: the deterministic continuation toward the MAP estimate, and the Euler step of the probability-flow ODE ($\beta = 1$).
- $h = 1$ jumps to the estimate and re-noises it: SDEdit-style restarts [6].

Any $\beta < 1$ re-injects noise, so the ascent then returns (approximately) a posterior sample rather than the MAP estimate.

The start. At $\sigma = \infty$, $r = 0$ forces $h = 1$. The first step returns $D_{\boldsymbol{c}}(\cdot, \infty) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{c}]$, whatever $\boldsymbol{u}$ is, and re-noises it: the ascent warm-starts itself from the posterior mean. An end-to-end reconstruction network (TGVN, LGGS) that regresses $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{y}, \boldsymbol{x}_1]$ in one shot is exactly that first step, without the rest.

5 Estimating the expectations

The algorithm consumes $D_{\boldsymbol{c}}$ and is indifferent to its source:

- $D_{\varnothing}$: a target-only denoiser, trained with the squared loss on $(\boldsymbol{u}, \boldsymbol{x}_0)$.
- $D_{\boldsymbol{x}_1}$: a guide-conditioned denoiser. Section 7 trains it on bridge samples.
- $D_{\boldsymbol{x}_1,\boldsymbol{y}}$, learned: a regressor that also sees $(\boldsymbol{y}, \boldsymbol{E})$ and the guide (SB-TGVN, BridgeLGGS).
- $D_{\boldsymbol{x}_1,\boldsymbol{y}}$ (or $D_{\boldsymbol{y}}$) from $D_{\boldsymbol{x}_1}$ (or $D_{\varnothing}$) by a **data step**. Approximate the posterior given $\boldsymbol{u}$ and the other variables by $\mathcal{N}(D, \varsigma^2 \boldsymbol{I})$ around the estimator's output D. With the Gaussian likelihood, the posterior mean given $\boldsymbol{y}$ too is the proximal point

$$(\boldsymbol{E}^H \boldsymbol{E} + \lambda \boldsymbol{I})^{-1} (\boldsymbol{E}^H \boldsymbol{y} + \lambda D), \qquad \lambda = \sigma_n^2 / \varsigma^2, \tag{7}$$

  solved by CG warm-started at D. The posterior variance is at most σ^2 on average ($\mathbb{E}\|\boldsymbol{x}_0 - D\|^2 = \mathrm{mmse}(\sigma) \le n\sigma^2$). So the principled weight follows the noise level, $\lambda(\sigma) = \sigma_n^2/(c\sigma^2)$ with $c \le 1$.

6 Case study: ImMAP2

ImMAP2 (`diffusion/immap.py`) has no guide. It is Algorithm 1 for the unguided posterior.

The objective. $\boldsymbol{c} = \{\boldsymbol{y}\}$, and the chain $\varnothing \subset \{\boldsymbol{y}\}$ has one step:

$$\log p(\boldsymbol{x}_0 \mid \boldsymbol{y}) = \log p(\boldsymbol{x}_0) + \log p(\boldsymbol{y} \mid \boldsymbol{x}_0) + \text{const}, \qquad \sigma^2 \nabla J_\sigma = \underbrace{(D_\varnothing - \boldsymbol{u})}_{\text{prior pull}} + \underbrace{(D_{\boldsymbol{y}} - D_\varnothing)}_{\text{measurement pull}}.$$

The estimator. Only $D_\varnothing$ is learned. The measurement increment comes from the data step (7) on $D_\varnothing$, solved by CG to tolerance, with

$$\lambda_k = \lambda_0 \, \sigma_n^2 \, \frac{1 + \sigma_k^2}{\sigma_k^2}.$$

This is (7) with $\varsigma^2 = \sigma_k^2/(1+\sigma_k^2)$, the posterior variance under a unit-variance Gaussian prior (the ΠGDM rule). It follows the noise level.

The schedule.

- **Self-paced ladder.** The level is re-estimated at every step from the residual, $\sigma_k^2 = \frac{1}{n}\|D_\varnothing(\boldsymbol{u}_k) - \boldsymbol{u}_k\|^2$. For an MMSE denoiser this is slightly low on average: $\mathbb{E}\|D_\varnothing - \boldsymbol{u}\|^2 = n\sigma^2 - \mathrm{mmse}$.
- **Step size** $h_k = h_0 k/(1 + h_0(k-1))$ with $h_0 = 0.01$, growing toward 1.
- **Injection** with $\beta = 0.05$, so $r = 1 - \beta h$: a slow contraction that samples.
- **Start** $\mathrm{mean}(\boldsymbol{E}^H \boldsymbol{y}) + \boldsymbol{z}$ at $\sigma = 1$.

The implementation returns the final state $\boldsymbol{u}$, not $\boldsymbol{v}$.

7 Specialising to a bridge

7.1 Why

Algorithm 1 needs the denoiser on the whole ladder, from very large (or infinite) σ down to 0. At large σ the input $\boldsymbol{u} = \boldsymbol{x}_0 + \sigma\boldsymbol{\varepsilon}$ is almost all noise:

- its norm grows like σ;
- the network must return $\approx \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{c}]$ while discarding all of it.

Standard diffusion copes in two ways. It rescales the input, as EDM's $c_{\text{in}}(\sigma)\boldsymbol{u}$ does [5]. And it stops at a finite $\sigma_{\max}$, paying for the mismatch between $\mathcal{N}(\boldsymbol{0}, \sigma_{\max}^2 \boldsymbol{I})$ and $p_{\sigma_{\max}}$. ImMAP2 sidesteps the problem by starting low, at $\sigma = 1$.

When the side information contains an image $\boldsymbol{x}_1$ close to $\boldsymbol{x}_0$, there is a better rescaling. Fix $V > 0$ and feed the network

$$\boldsymbol{x}_t = \boldsymbol{x}_1 + c(\sigma)\,(\boldsymbol{u} - \boldsymbol{x}_1), \qquad c(\sigma) = \frac{V}{\sigma^2 + V} = 1 - t, \qquad t = \frac{\sigma^2}{\sigma^2 + V} \in [0, 1]. \tag{8}$$

Proposition 2. *If $\boldsymbol{u} = \boldsymbol{x}_0 + \sigma\boldsymbol{\varepsilon}$, then $\boldsymbol{x}_t = (1-t)\,\boldsymbol{x}_0 + t\,\boldsymbol{x}_1 + \sigma_{\text{sb}}(t)\,\boldsymbol{\varepsilon}$ with $\sigma_{\text{sb}}(t)^2 = Vt(1-t)$. This is the Brownian-bridge marginal from $\boldsymbol{x}_0$ to $\boldsymbol{x}_1$, with $V = 4\tau^2$. Conversely,*

$$\boldsymbol{u} = \frac{\boldsymbol{x}_t - t\,\boldsymbol{x}_1}{1-t}, \qquad \sigma^2 = \frac{Vt}{1-t}. \tag{9}$$

For fixed $\boldsymbol{x}_1$ and $t < 1$ the map is a bijection. So $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t, \boldsymbol{x}_1] = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \boldsymbol{x}_1] = D_{\boldsymbol{x}_1}(\boldsymbol{u}, \sigma)$: a regressor trained on bridge samples is *the denoiser that Algorithm 1 needs for $\boldsymbol{c} = \{\boldsymbol{x}_1\}$.*

Proof. Substitute: $\boldsymbol{x}_1 + (1-t)(\boldsymbol{x}_0 + \sigma\boldsymbol{\varepsilon} - \boldsymbol{x}_1) = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1 + (1-t)\sigma\boldsymbol{\varepsilon}$, and $(1-t)^2\sigma^2 = (1-t)^2\,Vt/(1-t) = Vt(1-t)$. □

The level $\sigma = \sigma_{\text{sb}}/(1-t) = \sqrt{V}\sqrt{t/(1-t)}$ is a standard deviation. It diverges as $t \to 1$, although $\sigma_{\text{sb}} \to 0$, because σ_{sb} vanishes only like $\sqrt{1-t}$. (The finite quantity $\sigma_{\text{sb}}^2/(1-t) = Vt$ is the variance of the forward process before it is pinned at $\boldsymbol{x}_1$.) So bridge time is the smoothing level, from $\sigma = 0$ at $t = 0$ to $\sigma = \infty$ at $t = 1$.

Why it may work better. The bridge sample and the noisy target carry the same information, but the bridge sample is the much better network input:

- **Bounded noise.** The noise the network sees, $c(\sigma)\sigma = \sigma_{\text{sb}}(t)$, peaks at τ (at $t = \frac{1}{2}$) and vanishes at both ends, while σ runs from 0 to ∞ (Figure 1).
- **Bounded signal, exact start.** The input stays near the segment from $\boldsymbol{x}_0$ to $\boldsymbol{x}_1$. At $\sigma = \infty$ it is exactly $\boldsymbol{x}_1$, a finite input the network is trained on. So the top of the ladder is $\sigma = \infty$ with no start mismatch, and the first step of Algorithm 1 is the network's regression $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$.
- **A Wiener-filter preconditioner.** (8) is $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}]$ under the Gaussian prior $\boldsymbol{x}_0 \sim \mathcal{N}(\boldsymbol{x}_1, V\boldsymbol{I})$. So the network is fed the linear-Gaussian denoiser's output, and an $\boldsymbol{x}_0$-predicting network learns only the correction. The better $\boldsymbol{x}_1$ predicts $\boldsymbol{x}_0$, the smaller that correction.

In theory nothing is gained: the optimal outputs are equal. In practice a finite network has to fit the denoiser at every level, and the bridge removes the levels where that is hardest.

The bridge parameter τ encodes an assumed per-pixel spread $\sqrt{V} = 2\tau$ of $\boldsymbol{x}_0 - \boldsymbol{x}_1$. It shapes the preconditioner and the ladder, but not the target: $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \sigma, \boldsymbol{x}_1]$ does not depend on V. A guide of a different contrast fits the Gaussian assumption worse. That makes the correction larger; it does not change what the network should learn.

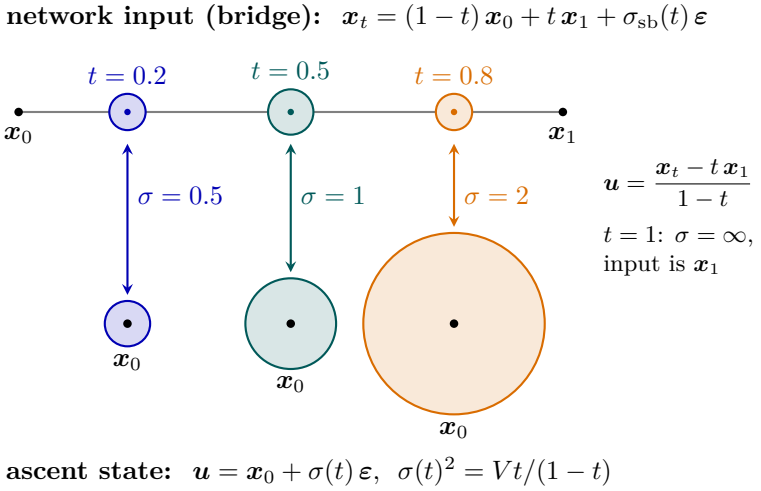


Figure 1: The same state in two coordinates. Bottom: the ascent state $\boldsymbol{u}$, the target $\boldsymbol{x}_0$ plus noise $\sigma(t)$, which grows without bound toward $t = 1$. Top: what the network is fed, the bridge sample (8), whose noise $\sigma_{\mathrm{sb}}(t)$ stays below τ and vanishes at both ends. The optimal outputs agree, $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t, \boldsymbol{x}_1] = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \boldsymbol{x}_1]$. Radii drawn to scale for $V = 1$.

7.2 Training

The bridge loss $\mathbb{E}\|D_\theta(\boldsymbol{x}_t, t, \boldsymbol{x}_1) - \boldsymbol{x}_0\|^2$, with $t \sim \mathcal{U}[0,1]$, is conditional denoising with the preconditioner (8). In EDM's vocabulary, only the design choices differ from a standard conditional diffusion:

	conditional VE diffusion	bridge
ideal denoiser	$\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \sigma, \boldsymbol{x}_1]$	the same
network input	$c_{\mathrm{in}}(\sigma)\,\boldsymbol{u}$, e.g. $(\sigma^2 + \sigma^2_{\mathrm{data}})^{-1/2}\boldsymbol{u}$	$\boldsymbol{x}_1 + c(\sigma)(\boldsymbol{u} - \boldsymbol{x}_1)$
training levels	e.g. log-normal σ	$p(\sigma) = 2\sigma V/(\sigma^2 + V)^2$
top level	finite $\sigma_{\max}$, start $\mathcal{N}(\mathbf{0}, \sigma^2_{\max}\boldsymbol{I})$	∞, start $\boldsymbol{x}_1$
sampling ladder	e.g. ρ-spaced in σ	uniform in t

The identity needs a network that is given $\boldsymbol{x}_1$. Without `cond_x1` (as in I^{2}SB-Inversion) it learns $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t]$. That network reads $\boldsymbol{x}_1$ only through $\boldsymbol{x}_t$, which works near $t = 1$ but is not the conditional denoiser in general.

Relation to unconditional diffusion. Since $\boldsymbol{u}$ and $\boldsymbol{x}_1$ are independent given $\boldsymbol{x}_0$, Bayes' rule splits the score, and Tweedie turns the split into denoisers:

$$\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}, \boldsymbol{x}_1] = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{u}] + \sigma^2\,\nabla_{\boldsymbol{u}} \log p_\sigma(\boldsymbol{x}_1 \mid \boldsymbol{u}). \tag{10}$$

This is the guide's PMI increment of Proposition 1, $D_{\boldsymbol{x}_1} - D_\varnothing$. A bridge learns it, where DPS [8] approximates it through the denoiser's Jacobian. It follows that a conditional-diffusion baseline on

the same $\boldsymbol{x}_1$ (SR3/Palette style) is not an independent method: same target, same sampler family. Any gap in results comes from the design choices in the table above.

8 Case study: I^{2}SB

The source image $\boldsymbol{x}_1$ is the only side information; there is no measurement.

The objective. $\boldsymbol{c} = \{\boldsymbol{x}_1\}$, and the chain $\varnothing \subset \{\boldsymbol{x}_1\}$ has one step:

$$\log p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1) = \log p(\boldsymbol{x}_0) + \log p(\boldsymbol{x}_1 \mid \boldsymbol{x}_0) + \text{const}, \qquad \sigma^2 \nabla J_\sigma = \underbrace{(D_\varnothing - \boldsymbol{u})}_{\text{prior pull}} + \underbrace{(D_{\boldsymbol{x}_1} - D_\varnothing)}_{\text{what } \boldsymbol{x}_1 \text{ adds}} = D_{\boldsymbol{x}_1} - \boldsymbol{u}.$$

A target that is plausible on its own and consistent with this particular source. I^{2}SB never splits the two pulls: its network estimates their sum, $D_{\boldsymbol{x}_1}$, directly.

8.1 The ascent I^{2}SB runs

Take Algorithm 1 with $\boldsymbol{c} = \{\boldsymbol{x}_1\}$ and three settings:

- the ladder $\sigma_k = \sigma(t_k)$ on a grid $1 = t_N > \cdots > t_0 = 0$, so $\sigma_N = \infty$;
- the ancestral step $h_k = 1 - r_k^2$, that is $\beta_k = 1/(1 + r_k)$;
- the denoiser read through Proposition 2, $D_{\boldsymbol{x}_1}(\boldsymbol{u}, \sigma) = D_\theta(\boldsymbol{x}_1 + c(\sigma)(\boldsymbol{u} - \boldsymbol{x}_1),\, t(\sigma),\, \boldsymbol{x}_1)$.

Write $t = t_k$, $s = t_{k-1}$. By (9), $r^2 = \sigma_s^2/\sigma_t^2 = \frac{s(1-t)}{t(1-s)}$, and the step is

$$\boldsymbol{u}_s = r^2\, \boldsymbol{u}_t + (1 - r^2)\, \boldsymbol{v} + \sigma_s \sqrt{1 - r^2}\, \boldsymbol{z}, \qquad \boldsymbol{v} = D_{\boldsymbol{x}_1}(\boldsymbol{u}_t, \sigma_t). \tag{11}$$

The step size is the variance removed, and the injection tops the leftover noise $r^2\sigma_t = r\sigma_s$ back up to exactly σ_s. This is `debiased_ascent` in `sb/i2sb_recon.py`.

8.2 Line by line into I^{2}SB

Map every state to the network's coordinates with (8), $\boldsymbol{x}_t = (1 - t)\, \boldsymbol{u}_t + t\, \boldsymbol{x}_1$. Algorithm 2 sets the two samplers side by side: each numbered line on the left becomes the same-numbered line on the right. Both run on the grid $1 = t_N > \cdots > t_0 = 0$, with $t = t_k$, $s = t_{k-1}$ and $r^2 = \sigma_s^2/\sigma_t^2 = \frac{s(1-t)}{t(1-s)}$, and both draw the same noise $\boldsymbol{z}_k$.

Algorithm 2 The same sampler in two coordinates

Ascent in $\boldsymbol{u}$ (Algorithm 1)		**I^{2}SB in $\boldsymbol{x}$** [2]
1: $\boldsymbol{u} \leftarrow$ anything	$\triangleright\ \sigma_N = \infty$	1: $\boldsymbol{x} \leftarrow \boldsymbol{x}_1$
2: **for** $k = N, \ldots, 1$ **do**		2: **for** $k = N, \ldots, 1$ **do**
3: $\quad \boldsymbol{v} \leftarrow D_\theta\big(\boldsymbol{x}_1 + (1-t)(\boldsymbol{u} - \boldsymbol{x}_1),\, t,\, \boldsymbol{x}_1\big)$		3: $\quad \hat{\boldsymbol{x}}_0 \leftarrow D_\theta(\boldsymbol{x},\, t,\, \boldsymbol{x}_1)$
4: $\quad \boldsymbol{u} \leftarrow r^2 \boldsymbol{u} + (1 - r^2)\, \boldsymbol{v} + \sigma_s \sqrt{1 - r^2}\, \boldsymbol{z}_k$		4: $\quad \boldsymbol{x} \leftarrow \frac{s}{t}\, \boldsymbol{x} + \big(1 - \frac{s}{t}\big) \hat{\boldsymbol{x}}_0 + \sqrt{V \frac{s(t-s)}{t}}\, \boldsymbol{z}_k$
5: **end for**		5: **end for**
6: **return** $\boldsymbol{u}$		6: **return** $\boldsymbol{x}$

No branches are needed at the ends:

- at $k = N$ ($t = 1$, $\sigma_t = \infty$, $r = 0$), line 4 is $\boldsymbol{u} = \boldsymbol{v} + \sigma_s \boldsymbol{z}_N$ on the left, and its image on the right;

- at $k = 1$ ($s = 0$, $\sigma_s = 0$, $r = 0$), line 4 returns the last estimate, $\boldsymbol{v}$ or $\hat{\boldsymbol{x}}_0$.

Proposition 3. *Each line of Algorithm 2 is the same operation in two coordinates. The right column is I²SB's reverse step [2]: the DDPM posterior of the bridge,* $\boldsymbol{x}_s \sim \mathcal{N}\big(\frac{s}{t}\boldsymbol{x}_t + (1-\frac{s}{t})\hat{\boldsymbol{x}}_0,\ V\frac{s(t-s)}{t}\boldsymbol{I}\big)$.

Proof. Line 1. At $t = 1$, $c = 0$, so $\boldsymbol{x}_t = \boldsymbol{x}_1$ for every $\boldsymbol{u}_N$. *Line 3.* The left network input is $\boldsymbol{x}_t$ by (8), so $\boldsymbol{v} = \hat{\boldsymbol{x}}_0$ (Proposition 2). *Line 4.* Multiply (11) by $1 - s$ and add $s\boldsymbol{x}_1$. The coefficients are

$$(1-s)\,r^2 = \tfrac{s(1-t)}{t}, \qquad (1-s)(1-r^2) = \tfrac{t-s}{t}.$$

Substituting $\boldsymbol{u}_t = (\boldsymbol{x}_t - t\boldsymbol{x}_1)/(1-t)$, the mean is

$$\tfrac{s}{t}(\boldsymbol{x}_t - t\boldsymbol{x}_1) + (1-\tfrac{s}{t})\hat{\boldsymbol{x}}_0 + s\boldsymbol{x}_1 = \tfrac{s}{t}\boldsymbol{x}_t + (1-\tfrac{s}{t})\hat{\boldsymbol{x}}_0.$$

The noise variance is

$$(1-s)^2\sigma_s^2(1-r^2) = (1-s)^2 \cdot \tfrac{Vs}{1-s} \cdot \tfrac{t-s}{t(1-s)} = V\tfrac{s(t-s)}{t}.$$

Line 6. At $t_0 = 0$, $\boldsymbol{x} = \boldsymbol{u}$. □

For a general I²SB schedule, replace t by $m_n = \sigma_{\text{fwd}}[n]^2/\sigma_{\text{fwd}}[N]^2$ and use $\sigma_n^2 = \sigma_{\text{fwd}}[N]^2\, m_n/(1-m_n)$; the proof is unchanged.

8.3 Numerical check

In the repository. `debiased_ascent` (`sb/i2sb_recon.py`) is the left column of Algorithm 2, and `reverse_sample` is the right. With the same seed they agree iterate for iterate (`tests/test_i2sb_recon.py`). The test uses a perturbed recon UNet, so the estimates are far from zero. The agreement is $2 \cdot 10^{-5}$ relative without the CG step and $2 \cdot 10^{-6}$ with 7 CG iterations.

On a trained network. A small image-to-image task, run with the script `equivalence_toy.py` in `notes/immap_i2sb`:

- **Task.** 4× super-resolution on CIFAR-10 (32×32, colour, in $[-1, 1]$). The target is the image. The start $\boldsymbol{x}_1$ is its 4× average-pooled version, upsampled bilinearly; the measurement is the pooling.
- **Network.** A 1.9M-parameter UNet given $\boldsymbol{x}_1$, trained with the bridge loss on 10k images for 20k steps. $\tau = 0.10$, so that $\sqrt{V} = 2\tau$ is the measured spread of $\boldsymbol{x}_0 - \boldsymbol{x}_1$ (0.20).

Both columns of Algorithm 2 were run with the same noise on 256 held-out images (`equivalence_toy_iterates.py`). After mapping $\boldsymbol{u} \mapsto \boldsymbol{x}$, every iterate agrees:

steps	worst iterate, relative difference	final PSNR, $\boldsymbol{u}$-ascent	final PSNR, I²SB
5	$7.6 \cdot 10^{-7}$	21.64 dB	21.64 dB
50	$7.8 \cdot 10^{-7}$	20.25 dB	20.25 dB

That is float32 rounding: the two algorithms are one computation. Figure 2 shows one such run in both coordinates.

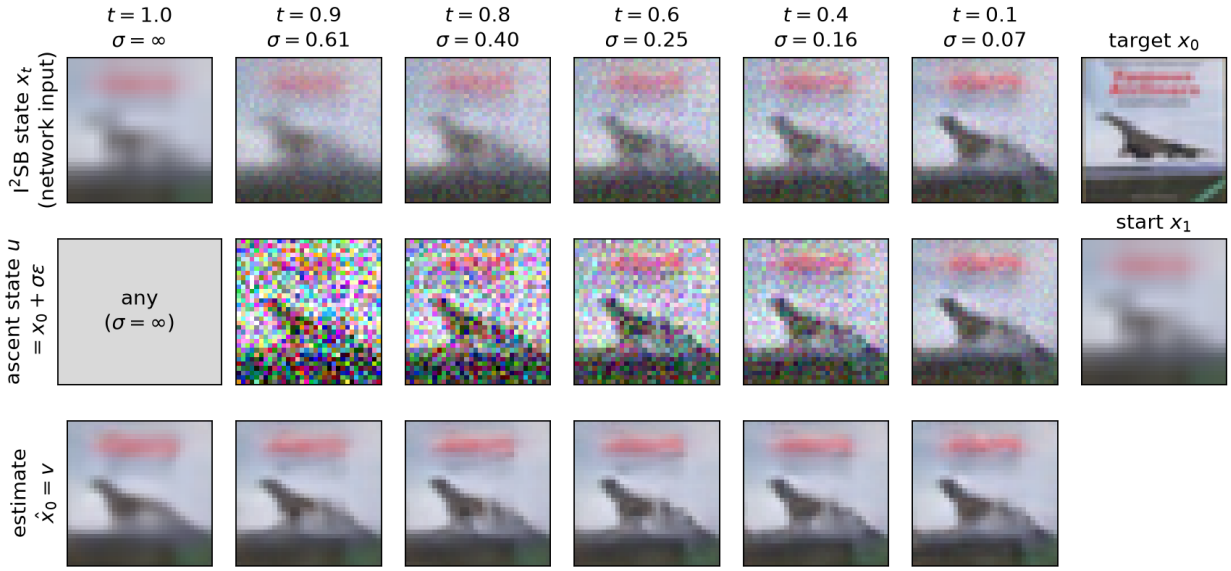


Figure 2: One I^2SB run (10 steps), shown in both coordinates of Algorithm 2, with the same noise draws. *Top:* the bridge state $\boldsymbol{x}_t$, which is what the network is fed; it starts at $\boldsymbol{x}_1$ and its noise never exceeds τ. *Middle:* the ascent state $\boldsymbol{u} = (\boldsymbol{x}_t - t\boldsymbol{x}_1)/(1-t)$, the target plus noise σ. It is arbitrary at $\sigma = \infty$, and saturated noise (clipped for display) at the first finite levels. *Bottom:* the estimate, identical in both coordinates. The first estimate is the one-shot regression $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$, and detail appears as σ falls.

The warm start is not needed. Algorithm 1 does not have to start at $\boldsymbol{x}_1$. The same trained network was run as the ascent from pure noise at $\sigma = 80$, on a $\rho = 7$ ladder in σ with $h = 1 - r^2$, and compared with native I^2SB. Each run draws 8 posterior samples for each of 512 held-out images, with 50 steps:

	sample PSNR	mean PSNR	consistency	posterior std	mean vs. I^2SB
I^2SB (start $\boldsymbol{x}_1$, $\sigma = \infty$)	20.30	21.99	0.0119	0.122	–
ascent from noise ($\sigma = 80$)	20.26	21.84	0.0131	0.122	0.92
I^2SB, network retrained (other seed)	20.19	21.90	0.0112	0.123	0.74

The columns are:

- **mean PSNR**: the PSNR of the 8-sample mean;
- **consistency**: the RMSE of the pooled sample against the measurement;
- **mean vs. I^2SB**: the RMS difference between posterior means, in units of the Monte-Carlo noise of a difference of two 8-sample means; about 1 means indistinguishable.

The two starts give the same posterior. Their means are as close as Monte-Carlo noise allows, and their spreads agree to 0.2%; retraining the network moves the result about as much. Starting at $\boldsymbol{x}_1$ is the $\sigma = \infty$ end of the same ascent, not a different algorithm. Figure 3 shows examples.

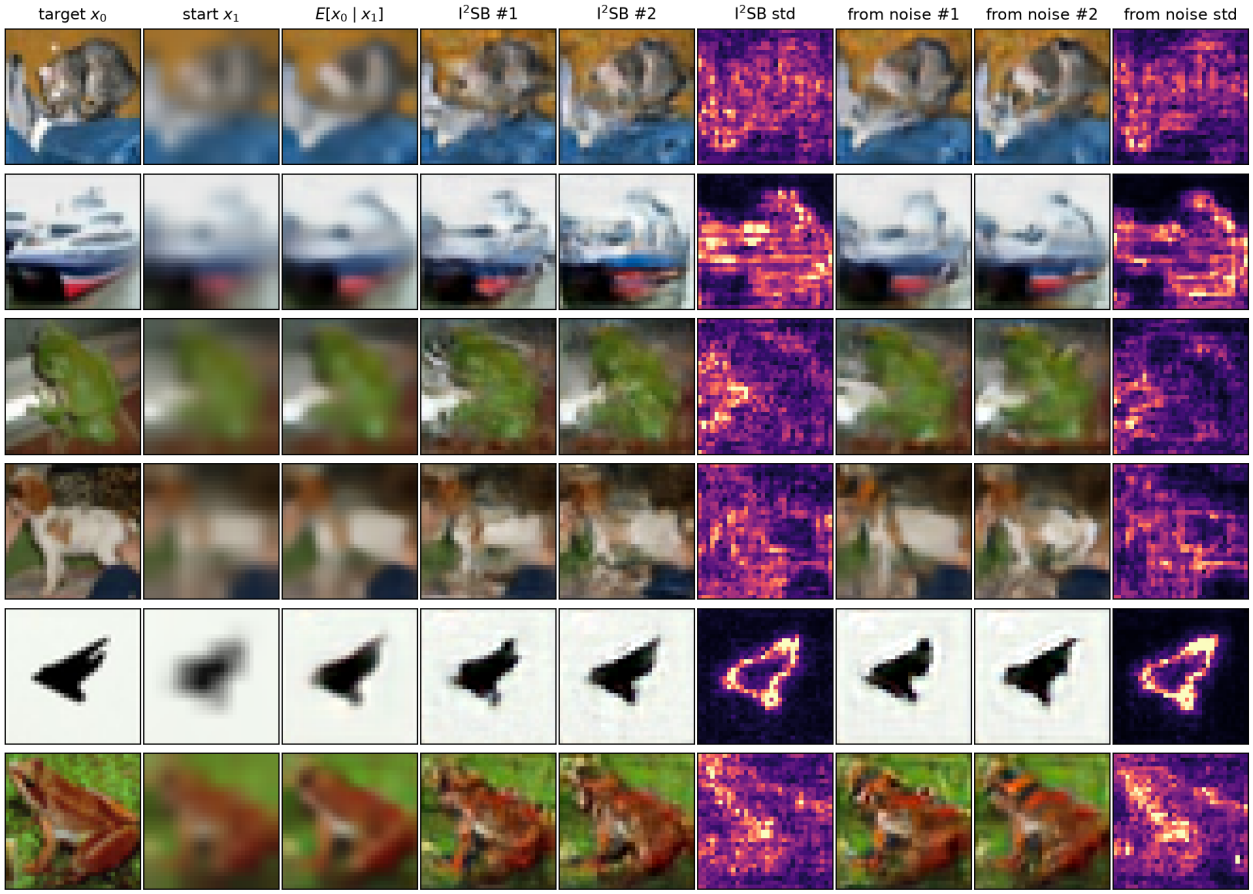


Figure 3: Held-out examples, 50 steps, from the same trained network. *Columns 1–3:* the target, the start $\boldsymbol{x}_1$, and the one-shot regression $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$ (the first step of either sampler), which is blurry. *Columns 4–6:* two I^{2}SB samples (start $\boldsymbol{x}_1$, $\sigma = \infty$) and the per-pixel standard deviation over 8 samples. *Columns 7–9:* the same for the ascent started at pure noise ($\sigma = 80$). The two starts produce samples of the same character, and their uncertainty concentrates on the same edges and textures that the low-resolution start cannot resolve.

8.4 A run, from $\boldsymbol{x}_1$ to $\boldsymbol{x}_0$

- **First step** ($t_N = 1$, $\boldsymbol{x}_t = \boldsymbol{x}_1$, $\sigma = \infty$, $r = 0$, $h = 1$). The state carries no information beyond $\boldsymbol{x}_1$, so the network returns the posterior mean $\hat{\boldsymbol{x}}_0 = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$: a blurry average of every target consistent with $\boldsymbol{x}_1$. The step jumps to it and re-noises: $\boldsymbol{x}_{t_{N-1}} = (1 - t_{N-1})\,\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1] + t_{N-1}\boldsymbol{x}_1 + \sigma_{\text{sb}}(t_{N-1})\boldsymbol{z}$. This is a warm start at level $\sigma(t_{N-1})$, as in SDEdit [6] and come-closer-diffuse-faster [7], not "starting at $\boldsymbol{x}_1$".
- **Middle steps** ($0 < h < 1$). Each moves the state the fraction $1 - r^2$ toward the current estimate and re-injects noise. The fresh noise commits the run to one of the targets the estimate was averaging over; the next estimate, at lower σ, is sharper. Unrolled, the state is a precision-weighted average of all estimates so far.
- **Last step** ($s = 0$, $\sigma = 0$, $r = 0$, $h = 1$). No noise is injected, and the output is the final estimate.

step $t \to s$	$\sigma(t)$	$\sigma(s)$	r	$h = 1 - r^2$	β	injected (in $\boldsymbol{u}$)	$\boldsymbol{x}_s$: weight on $\boldsymbol{x}_t$, $\hat{\boldsymbol{x}}_0$	noise (in $\boldsymbol{x}$)
$1 \to 0.75$	∞	0.658	0	1	1	0.658	0.75, 0.25	0.165
$0.75 \to 0.5$	0.658	0.380	0.577	0.667	0.634	0.310	0.67, 0.33	0.155
$0.5 \to 0.25$	0.380	0.219	0.577	0.667	0.634	0.179	0.50, 0.50	0.134
$0.25 \to 0$	0.219	0	0	1	1	0	0, 1	0

Table 1: Four uniform steps at $\tau = 0.19$ ($V = 0.144$). At the first step the weight 0.75 falls on $\boldsymbol{x}_t = \boldsymbol{x}_1$, so $\boldsymbol{x}_{t_{N-1}} = 0.25\,\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1] + 0.75\,\boldsymbol{x}_1 + 0.165\,\boldsymbol{z}$. Uniform steps in t give equal ratios r in the middle of the ladder.

A sample, not the MAP estimate. $\beta = 1/(1+r) < 1$ re-injects noise, so I^{2}SB returns (approximately) a sample of $p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)$. The approximation is the point estimate: the exact reverse step would also carry the spread $(1-r^2)^2\,\mathrm{Cov}[\boldsymbol{x}_0 \mid \boldsymbol{u}_t, \boldsymbol{x}_1]$ of the endpoint, which vanishes on fine ladders.

- The same network gives the deterministic continuation toward the MAP estimate with $h = 1 - r$ and no injection.
- Keeping $h = 1 - r^2$ and only dropping the noise (`deterministic=True`) is neither: the state then lands at noise $r^2\sigma_t < \sigma_s$, below the level the next call is told.

What the $\boldsymbol{u}$-form frees. I^{2}SB fixes $h = 1 - r^2$, the uniform t-grid and $\sigma_N = \infty$. Algorithm 1 accepts the same trained network with any $h \in [1-r, 1]$, any ladder (including ImMAP2's self-paced one), or a finite-σ start from noise. On a fixed t-grid every σ_k scales with $\sqrt{V} = 2\tau$, so τ decides how warm the start is: at 20 uniform steps, $\sigma(0.95) \approx 1.7$ for $\tau = 0.19$ and ≈ 0.8 for $\tau = 0.09$, against image intensities ~ 1.

8.5 Few steps, other starts, and a self-paced ladder

The CIFAR task of Section 8.3 tests those freedoms. Every run draws 8 samples for each of 512 held-out images, at a budget of 2 to 50 network calls (NFE, counting every call). Three measures:

- **consistency**: the RMSE of the pooled sample against the measurement;
- **detail**: the energy of the band the measurement cannot see, $\|\boldsymbol{x} - \boldsymbol{x}_1(\boldsymbol{x})\|^2$, relative to the target's. The one-shot regression has 0.29; a perfect posterior sample would have 1;
- **posterior std**: the spread over the 8 samples.

Single-sample PSNR is not used. It *rises* as steps are removed, because the samples collapse toward the posterior mean (the regression alone scores 22.3 dB, against 20.3 dB for 50-step I^{2}SB).

Besides the bridge network, the comparison uses a second network: a standard conditional diffusion network with EDM preconditioning [5]. It has the same architecture, data, $\boldsymbol{x}_1$-conditioning and budget, and is trained on log-normal σ. Its estimates at $\sigma = 80$ are 33% worse than the bridge network's, which is flat all the way to $\sigma = \infty$. At $\sigma \le 0.05$ it is better, thanks to its skip connection.

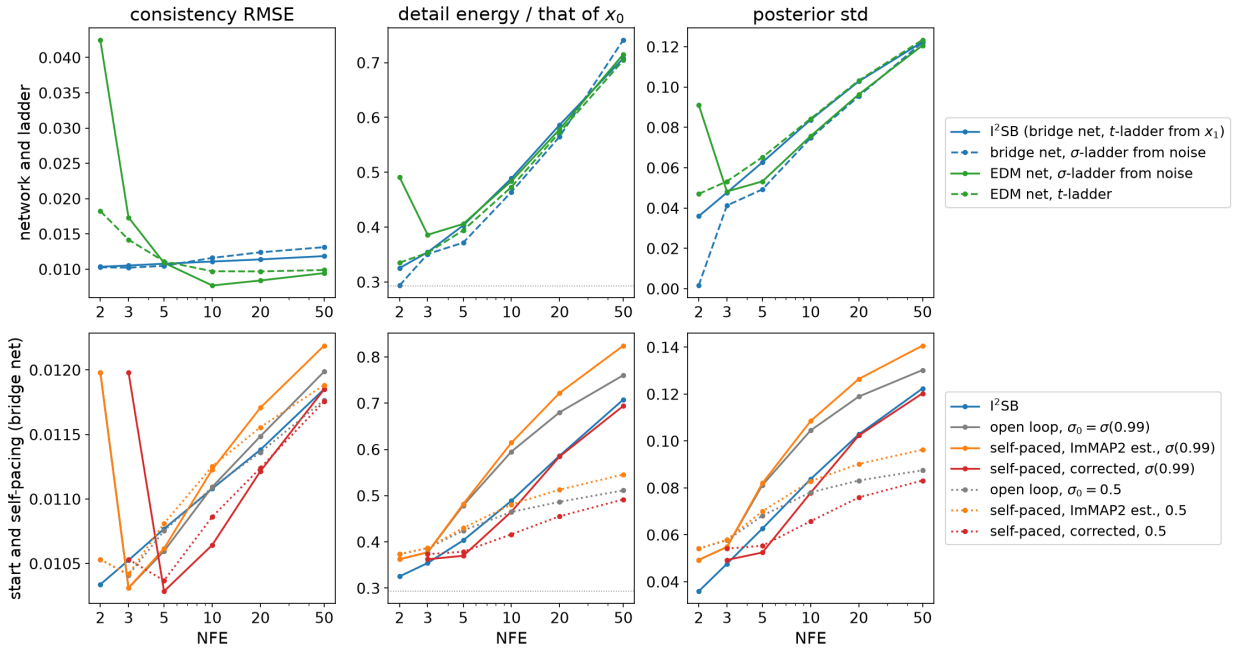


Figure 4: Step-count sweep on the CIFAR task. *Top:* the two networks on the two ladders: I^{2}SB's t-grid from $\boldsymbol{x}_1$, and a $\rho = 7$ σ-ladder from noise at $\sigma = 80$. *Bottom:* the bridge network with a different start and ladder. It regresses at $\sigma = \infty$, re-noises to σ_0, then follows geometric target levels, either open loop or self-paced (Section 8.5). The dotted line marks the one-shot regression's detail. Note the narrow range of the bottom-left panel.

Few steps collapse toward the regression. With 2 steps, every sampler returns close to $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$: I^{2}SB's detail is 0.33 (the regression has 0.29) and its spread is 0.036 (against 0.122 at 50 steps). Detail and spread grow with the step count and have not saturated at 50 steps, where samples carry 0.71 of the target's detail.

The bridge pays off in the first steps.

	consistency ($\times 10^{-3}$), NFE				detail, NFE	
	2	3	5	50	2	50
I^{2}SB (bridge net, t-ladder from $\boldsymbol{x}_1$)	10.3	10.5	10.8	11.9	0.33	0.71
bridge net, σ-ladder from noise	10.3	10.2	10.5	13.1	0.29	0.74
EDM net, σ-ladder from noise	42.5	17.3	10.9	9.4	0.49	0.72
EDM net, t-ladder	18.2	14.2	11.1	9.9	0.34	0.70

- **With 2 steps the EDM network fails.** From noise, its consistency error is 4× the bridge network's, and its high detail (0.49) is leftover noise: its first estimate, at $\sigma = 80$, is poor. I^{2}SB's ladder helps it ($18 \cdot 10^{-3}$) but does not close the gap.
- **The bridge network is equally good from $\boldsymbol{x}_1$ or from noise.** It is accurate up to $\sigma = \infty$, so its first estimate is right whatever the start.
- **From 5 steps on, the curves merge.** Past 10 steps the EDM network is slightly more consistent, through its advantage at small σ.

This is Section 7's conditioning argument in practice. The bridge input matters where the high-σ estimates matter, which is when there are few steps. With the bridge network, the ladder itself matters little: I^{2}SB's t-grid and the σ-ladder from noise nearly overlap in every panel.

Self-pacing does not help at matched NFE. The self-paced ladder:

1. regresses at $\sigma = \infty$;
2. re-noises to a start σ_0;
3. at each step, estimates the level actually present and steps from it to the next target level, with $h = 1 - r^2$ and $r = \sigma_{\text{target}}/\hat{\sigma}$;
4. tells the network $\hat{\sigma}$.

The level estimate is either ImMAP2's, $\hat{\sigma}^2 = \|D - \boldsymbol{u}\|^2/n$, or the corrected $\hat{\sigma}^2 = \|D - \boldsymbol{u}\|^2/(n - \operatorname{div} D)$. The correction follows from $\mathbb{E}\|D - \boldsymbol{u}\|^2 = n\sigma^2 - \text{mmse}$ and $\text{mmse} = \sigma^2\mathbb{E}[\operatorname{div} D]$, and costs one extra call per step. The open-loop control uses the same start and targets with no estimate.

start σ_0	ladder	first $\hat{\sigma}/\sigma_0$	NFE 10: std, detail		NFE 50: std, detail	
$\infty \to \sigma(0.98)$	I^{2}SB	–	0.084	0.49	0.122	0.71
$\sigma(0.99) \approx 2.0$	open loop	–	0.105	0.60	0.130	0.76
	self-paced, ImMAP2 estimate	0.995	0.109	0.61	0.141	0.82
	self-paced, corrected	0.998	0.078	0.47	0.120	0.69
0.5	open loop	–	0.078	0.46	0.088	0.51
	self-paced, ImMAP2 estimate	0.971	0.083	0.48	0.096	0.55
	self-paced, corrected	0.989	0.066	0.42	0.083	0.49

- **ImMAP2's estimate reads low**, more so at lower σ (0.995 at $\sigma_0 = 2.0$, 0.971 at 0.5); the correction removes most of the bias. A low $\hat{\sigma}$ makes each step remove too little noise. The extra spread and "detail" at 50 steps (0.141, 0.82, above the open loop's 0.130, 0.76) are leftover noise.
- **The corrected estimate tracks I^{2}SB**, at about half the steps per NFE: at 50 NFE it has 24 steps and matches 50-step I^{2}SB's spread and detail. At matched NFE it gains nothing over the open loop or I^{2}SB.
- **The start level matters more than the pacing.** From $\sigma_0 = 0.5$, about three times the regression's RMS error, all three variants plateau near the regression: spread about 0.09 and detail about 0.5, even at 50 NFE. That noise is too little to leave the neighbourhood of $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_1]$ (the SDEdit trade-off). From $\sigma(0.99)$ the variants reach I^{2}SB's spread. I^{2}SB's own first re-noise, at 50 steps, is $\sigma(0.98) \approx 1.4$.

Limits: one task, small networks, and no perceptual metric. Detail has not saturated at 50 steps, so these samplers are compared short of convergence.

9 Adding the measurement: I^{2}SB-Recon and ImMAP2

I^{2}SB-Recon and I^{2}SB-Inversion [3]. The guide is a fully sampled scan of *another contrast* (T1 for a T2 target; PD for PDFS here), and it is the bridge start, $\boldsymbol{x}_1 = \text{guide}$. The objective is the full posterior, $\boldsymbol{c} = \{\boldsymbol{x}_1, \boldsymbol{y}\}$:

$$\log p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y}) = \log p(\boldsymbol{x}_0) + \log p(\boldsymbol{x}_1 \mid \boldsymbol{x}_0) + \log p(\boldsymbol{y} \mid \boldsymbol{x}_0) + \text{const.}$$

- The network estimates only $D_{\boldsymbol{x}_1}$; it never sees $\boldsymbol{y}$.
- The measurement term is added by the data step (7) on $D_{\boldsymbol{x}_1}$, with $\lambda = 7 \cdot 10^{-5}$ fixed and 7 CG iterations, giving $\boldsymbol{v} \approx D_{\boldsymbol{x}_1,\boldsymbol{y}}$. The data step is the only place the measurement enters.
- The schedule is I^{2}SB's (11), with 80 steps.
- I^{2}SB-Inversion then walks the reconstruction back up the bridge deterministically, and runs the same ascent a second time from there.

The step is ImMAP2's. ImMAP2 at level σ_k denoises, applies (7), then takes

$$\boldsymbol{u}_{k+1} = \boldsymbol{u}_k + h_k(\bar{\boldsymbol{x}}_0 - \boldsymbol{u}_k) + \sigma_k\sqrt{(1-\beta h_k)^2 - (1-h_k)^2}\,\boldsymbol{z}, \qquad \sigma_{k+1} = (1-\beta h_k)\,\sigma_k, \tag{12}$$

where $\bar{\boldsymbol{x}}_0$ is the data step's output.

Proposition 4. *With $r = \sigma_{k+1}/\sigma_k$, $h_k = 1 - r^2$ and $\beta = 1/(1+r)$, the ImMAP2 step (12) is the I^{2}SB step (11) with $\boldsymbol{v}$ replaced by $\bar{\boldsymbol{x}}_0$: the I^{2}SB-Recon step, written in the coordinates (9).*

Proof. $1-\beta h_k = (1+r-1+r^2)/(1+r) = r$, so the next level is σ_{k+1}. The mean $\boldsymbol{u}_k + (1-r^2)(\bar{\boldsymbol{x}}_0 - \boldsymbol{u}_k)$ is that of (11), and the noise variance is $\sigma_k^2(r^2 - r^4) = \sigma_{k+1}^2(1-r^2)$. □

So the two methods share the step structure exactly. Their settings differ:

	ImMAP2	I^{2}SB-Recon
objective	$p(\boldsymbol{x}_0 \mid \boldsymbol{y})$	$p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y})$
denoiser	unconditional, $D_\varnothing$	guide-conditioned, $D_{\boldsymbol{x}_1}$
network input	$\boldsymbol{u}$	bridge sample (8)
noise ladder	self-paced, $\sigma_k^2 = \frac{1}{n}\lVert D_\varnothing - \boldsymbol{u}_k\rVert^2$	fixed t-grid
step size h_k	$h_0 k/(1 + h_0(k-1))$	$1 - r^2$, the variance removed
β	fixed, 0.05	$1/(1+r)$: 1 on the first step, $\to \frac{1}{2}$
prox weight λ	$\lambda_0\sigma_n^2(1+\sigma_k^2)/\sigma_k^2$	constant $(7 \cdot 10^{-5})$
start	mean$(\boldsymbol{E}^H\boldsymbol{y}) + \boldsymbol{z}$ at $\sigma = 1$	$\boldsymbol{x}_1$ at $\sigma = \infty$: regress, re-noise

Remark 1 (ImMAP2 and I^{2}SB-Recon are not comparable as such). Strictly, the two solve different problems. I^{2}SB-Recon is given the guide and ImMAP2 is not, so the first row of the table differs: $p(\boldsymbol{x}_0 \mid \boldsymbol{x}_1, \boldsymbol{y})$ against $p(\boldsymbol{x}_0 \mid \boldsymbol{y})$. A gap in results conflates the extra information with the estimator and the schedule. Giving ImMAP2 the guide means replacing $D_\varnothing$ by $D_{\boldsymbol{x}_1}$; giving I^{2}SB-Recon ImMAP2's λ_k isolates the data weight.

With a *zero-filled* start, $\boldsymbol{x}_1 = \boldsymbol{E}^H\boldsymbol{y}$ (our `sbunet` baseline), the objectives coincide. Under Gaussian noise $\boldsymbol{E}^H\boldsymbol{y}$ is sufficient for $\boldsymbol{y}$:

$$\log p(\boldsymbol{y} \mid \boldsymbol{x}_0) = \tfrac{1}{\sigma_n^2}\,\mathrm{Re}\langle \boldsymbol{E}^H\boldsymbol{y}, \boldsymbol{x}_0\rangle - \tfrac{1}{2\sigma_n^2}\lVert \boldsymbol{E}\boldsymbol{x}_0\rVert^2 + \text{const.}$$

So $\{\boldsymbol{x}_1, \boldsymbol{y}\}$ carries the same information as $\{\boldsymbol{y}\}$, and $D_{\boldsymbol{x}_1} = D_{\boldsymbol{x}_1,\boldsymbol{y}} = D_{\boldsymbol{y}}$. Then I^{2}SB-Recon is, in theory, ImMAP2 with a different estimator of the same $D_{\boldsymbol{y}}$ and different settings. The data step adds no information in exact arithmetic and only corrects the network's errors.

In practice they need not agree. The estimators differ: a learned regression of $D_{\boldsymbol{y}}$ against a Gaussian-approximation data step on $D_\varnothing$. And so do the input coordinates, the ladder, h, β, λ and the start.

10 The methods as instances

Every method below is Algorithm 1. What differs is the conditioning set (what is estimated), the estimator, and the schedule: noise ladder, h_k, β, start.

method	conditions on	estimator of $D_{\boldsymbol{c}}$	ladder, h, β
SDEdit	nothing	$D_\varnothing$; $\boldsymbol{x}_1$ only in the start $\boldsymbol{x}_1 + \sigma_0 \boldsymbol{z}$	free
ImMAP2	$\boldsymbol{y}$	data step on $D_\varnothing$, $\lambda_k \propto (1 + \sigma_k^2)/\sigma_k^2$	self-paced; KS h_k; $\beta = 0.05$
I^{2}SB	guide	$D_{\boldsymbol{x}_1}$ (bridge regressor)	fixed ladder; $h = 1 - r^2$; $\beta = \frac{1}{1+r}$
I^{2}SB-Recon / -Inversion	guide, $\boldsymbol{y}$	data step on $D_{\boldsymbol{x}_1}$, $\lambda = 7 \cdot 10^{-5}$	as I^{2}SB, 80 steps
SB-TGVN, BridgeLGGS (any start)	guide, $\boldsymbol{y}$	learned $D_{\boldsymbol{x}_1, \boldsymbol{y}}$ (the net sees both)	as I^{2}SB
`sbunet` (zero-filled start)	$\boldsymbol{y}$	$D_{\boldsymbol{x}_1}$ with $\boldsymbol{x}_1 = \boldsymbol{E}^H \boldsymbol{y}$, + data step	as I^{2}SB
TGVN, LGGS end-to-end	guide, $\boldsymbol{y}$	learned $\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{y}, \boldsymbol{x}_1]$	one step from $\sigma = \infty$

Table 2: Every method is Algorithm 1. The first two columns say what is estimated; the last is how the ascent is run.

For SB-TGVN the bridge start is a parameterisation, not an objective. Whatever $\boldsymbol{x}_1$ is, the net conditions on the guide and $\boldsymbol{y}$. So the three starts target the same posterior and differ only in the path.

References

[1] Z. Kadkhodaie, E. P. Simoncelli. Stochastic solutions for linear inverse problems using the prior implicit in a denoiser. *NeurIPS*, 2021.

[2] G.-H. Liu, A. Vahdat, D.-A. Huang, E. A. Theodorou, W. Nie, A. Anandkumar. I^{2}SB: Image-to-Image Schrödinger Bridge. *ICML*, 2023.

[3] Y. Wang et al. Guided MRI reconstruction via Schrödinger bridge. arXiv:2411.14269, 2024.

[4] Y. Song et al. Score-based generative modeling through stochastic differential equations. *ICLR*, 2021.

[5] T. Karras et al. Elucidating the design space of diffusion-based generative models. *NeurIPS*, 2022.

[6] C. Meng et al. SDEdit. *ICLR*, 2022.

[7] H. Chung, B. Sim, J. C. Ye. Come-closer-diffuse-faster. *CVPR*, 2022.

[8] H. Chung et al. Diffusion posterior sampling for general noisy inverse problems. *ICLR*, 2023.